

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4538 – IOS İLE MOBİL UYGULAMA
GELİŞTİRME II

KAMU KÖPRÜSÜ

Aybaran Yurtseven - 22290067

GitHub: <https://github.com/aybaranyurtseven>

İçindekiler

1	GİRİŞ	2
1.1	Kamu Köprüsü Nedir?	2
1.2	Amaç ve Kapsam	2
2	PROJE ORTAMI VE KULLANILACAK TEKNOLOJİLER	4
2.1	Geliştirme Ortamı	4
2.2	Kullanılacak Yazılım Teknolojileri	4
2.3	Sistem Mimarisi Düşüncesi	5
3	YAZILIM GERÇEKLEME SÜRECİ	7
3.1	Geliştirilecek Modüller ve Ekranlar (Screens)	7
3.1.1	Kimlik İşlemleri Modülü (Auth Stack)	7
3.1.2	Vatandaş Deneyimi Modülü (Citizen Stack)	7
3.1.3	Kurum ve Moderatör Paneli Modülü	8
3.2	Veri Tabanı Şeması Tasarımı	8

1 GİRİŞ

Günümüzde akıllı cihaz kullanımının hızla yaygınlaşmasıyla, vatandaşların kamu kurumlarıyla iletişiminin mobil uygulamalar üzerinden anlık olarak sağlanması önemli bir ihtiyaç haline gelmiştir. Geleneksel başvuru mekanizmalarının yetersiz ve bürokratik yanlarını gidermek amacıyla planlanan "Kamu Köprüsü" projesi, mobil platformun avantajlarını kullanarak hızlı, pratik ve şeffaf bir başvuru ağı oluşturmayı hedeflemektedir.

1.1 Kamu Köprüsü Nedir?

Kamu Köprüsü, vatandaşların kamu kurumlarına yönelik şikayet, öneri veya tebriklerini doğrudan cep telefonlarından fotoğraf, video veya ses dosyalarıyla kanıtlandırarak iletebilecekleri ve başvurularının güncel durumunu anlık olarak takip edebilecekleri bir m-devlet portalı olarak tasarlanmaktadır.

Geliştirilecek uygulamanın sağlayacağı başlıca avantajlar şunlar olacaktır:

- Vatandaşlara mekan bağımsız, bürokrasiden uzak ve hızlı bir başvuru imkanı sunması.
- Cihaz donanımları (kamera, konum GPS) kullanılarak şikayeti anlık ve doğrudan medya yüklemeleriyle desteklemesi.
- Uygulama içi mobil bildirimler (Push Notifications) ile çözüm süreçlerinin takibinin daha şeffaf hale getirilmesi.
- Puanlama mekanizması, ilerleme çubuğu ve rozet kazanımı gibi oyunlaştırma (gamification) elementleriyle sivil katılımın teşvik edilmesi.

1.2 Amaç ve Kapsam

Projenin temel amacı, klasik ve hantal web formları üzerinden ilerleyen şikayet süreçlerini mobil dünyanın dinamik ve reaktif ortamına taşımaktır. Yalnızca web versiyonu oluşturmak yerine cebimizde taşıdığımız bir portal üretebilmek hedeflenmektedir. Proje kapsamında aşağıdaki uygulamaların gerçekleştirilmesi planlanmaktadır:

- **Çok Rollü Yönetim Sistemi:** Vatandaş, Kurum Görevlisi ve Moderatör gibi rollere özel fonksiyonlara sahip, sadeleştirilmiş özel mobil ekran arayüzlerinin oluşturulması.

- **Anlık Şikayet Akışı:** Doğrudan cihaz klasörlerinden veya kamerasından görsel medya dökümanları çekip bulut tabanlı NoSQL veritabanı (Firestore) üzerinde açılan şikayet belgelerine anında dahil eden kesintisiz bir sistemin kurulması.
- **Zahmetsiz Kimlik Doğrulama:** Sisteme e-posta/şifre tabanlı veya popüler sağlayıcılar ile güvenli kimlik doğrulama süreçlerinin (Firebase Auth) entegre edilmesi.

2 PROJE ORTAMI VE KULLANILACAK TEKNOLOJİLER

Bu bölümde projenin hayata geçirilmesinde kullanılacak olan geliştirme ortamı, araçlar ve performans odaklı seçilen mobil teknoloji yığıtı (Tech Stack) detaylandırılmaktadır.

2.1 Geliştirme Ortamı

Mobil uygulamanın yazım süreci ve hata ayıklamaları ağırlıklı olarak Visual Studio Code gibi modern ve eklenti yönünden zengin kod editörleri üzerinde sürdürülecektir. Yazılım geliştirme sürecinin anlık testleri ve Android/iOS simülatör önizlemeleri için derleme (build) maliyetini büyük ölçüde düşüren ve geliştirme hızını artıran Expo framework'ü kullanılacaktır. Kodun versiyonlanması ve sürüm yönetimi süreçleri ise Git altyapısı ile GitHub platformu üzerinde koordineli şekilde yönetilecektir.

2.2 Kullanılacak Yazılım Teknolojileri

Modern mobil platformlarda yüksek performansla kod geliştirmek için proje mimarisi aşağıdaki esnek, güncel ve yaygın destekleyici teknolojiler etrafında kurgulanacaktır:

- **React Native:** Hem Android hem de iOS odaklı, cihazın kendi çekirdek dillerine en yakın (native) performansta mobil uygulamalar geliştirmek için ana teknoloji olarak kullanılacaktır. JavaScript / TypeScript kod tabanıyla iki işletim sistemi için de ortak kod yazımına imkan tanıyarak geliştirme eforunu ve maliyetini büyük ölçüde azaltacaktır.
- **React Navigation:** Uygulama içi akıcı yönlendirmeler (Routing) için tercih edilecektir. Stack (üst üste geçen ekranlar), Bottom Tab (alt sekme menüsü) ve Drawer (yan menü) navigasyon yapıları profesyonel olarak kullanılarak ekranlar arası hızlı, doğal hissettiren (native-feeling) geçişler elde edilecektir. Ekranla bağlı parametre veri transferleri de bu kütüphane aracılığıyla gerçekleştirilecektir.
- **Redux Toolkit & React Context API:** Uygulamanın durum yönetimi (State Management) ihtiyaçlarını karşılamak için kurgulanacaktır. Kullanıcı oturum bilgisi, alınan şikayet listelerinin filtrelenmiş verileri veya uygulama içi karanlık/açık tema tercihleri global bir deponun (Store) içerisinde tutulacaktır. Böylece propların manuel olarak taşınması

zahmetinden kurtululacak ve herhangi bir bileşenden global verilere anlık, reaktif erişim sağlanacaktır.

- **Axios / Fetch API:** Firebase dışındaki olası harici RESTful veri servislerine (örneğin hava durumu, konum tabanlı ters coğrafi kodlama -Reverse Geocoding- servisleri veya kamu bilgi API'ları) yapılacak olan asenkron HTTP isteklerini yönetmek için kullanılacaktır. Gelen API yanıtları asenkron yapı sayesinde kullanıcı arayüzünü (UI thread) kilitlemeden arka planda sorunsuz çözümlenecektir.
- **Firestore Authentication:** Kullanıcıların sisteme güvenli kayıt, şifreli giriş ve parola sıfırlama operasyonları için yüksek güvenli, hazır bulut kimlik doğrulama altyapısı olarak entegre edilecektir. E-posta/şifre tabanlı standart girişin yanında istenirse gelecekte Google veya Apple ile giriş gibi Sosyal Yetkilendirme (Social Login) yeteneklerine de şimdiden zemin hazırlayacaktır. Oturum biletleri (JWT) bu teknoloji sayesinde güvenli yönetilecektir.
- **Firestore (NoSQL Veri Tabanı):** Son derece esnek tasarımı, bulut tabanlı bir NoSQL veri tabanı olan Firestore ana sunucu deposu olarak kullanılacaktır. Şikayetlerin, platformdaki kullanıcı profillerinin ve kurum verilerinin hiyerarşik Koleksiyon/Doküman (Collection/Document) mimarisinde, JSON mimarisine benzer bir formatta tutulması sağlanacaktır. Klasik ilişkisel yavaş veritabanlarının (SQL) aksine güçlü gerçek zamanlı anlık veri senkronizasyon (real-time sync) yeteneği sayesinde bildirimler ve veri güncellemeleri anında tüm kullanıcı cihazlarına (istemcilere) iletilebilecektir.
- **Firestore Storage:** Kullanıcılar tarafından şikayet kanıtı veya görsel zenginlik olarak uygulamanın içine yüklenecek olan fotoğraf, video veya ses dosyaları gibi büyük gövdeli medya içeriklerinin esnek bulut depolama sunucularında barındırılması için kullanılacaktır. Veri tabanının şişmesinin önüne geçmek amacıyla, Storage'a yüklenen dosyaların sadece onlara ait dış iznin olduğu indirme URL referansları (download URLs) Firestore üzerinde metin olarak tutulacaktır.

2.3 Sistem Mimari Düşüncesi

Projenin genel tasarım dilinde ve kod mimarisinde Bileşen Odaklı Mimari (Component-based architecture) kalıpları çerçevesinde ilerlenecektir. Uygulama içerisindeki karmaşık arayüzler;

şık tasarımı dokunmatik butonlar, zengin medya listeleri, metin giriş alanları veya uyarı pencereleri (modals) gibi tekrar eden ve uygulamanın pek çok yerinde işlev gören izole küçük kod bileşenlerine (komponentlere) bölünecektir. Bileşenlerin servis ve veritabanıyla arasındaki sıkı arka uç (backend) diyalogu özel çağırıcı kancalar (Custom React Hooks) içine soyutlanarak, böylece kodun okunabilirliği artırılacak ve bakımı çok daha profesyonel bir seviyeye çekilecektir.

3 YAZILIM GERÇEKLEME SÜRECİ

Geliştirme fazlasının verimli ilerleyebilmesi için öncelikle arayüz (Ekran UI/UX) tasarımlarının React bileşenlerine dönüştürülmesi, hemen ardından ise veritabanında yapısal ihtiyaçların (NoSQL) kurgulanması benimsenecektir. Sistem işlevselliğine göre aşağıdaki ana sayfalar kümesine (Stack) ayrılacaktır:

3.1 Geliştirilecek Modüller ve Ekranlar (Screens)

Sistem işlevselliğine göre belirli ekran kümelenmelerine ayrılacaktır:

3.1.1 Kimlik İşlemleri Modülü (Auth Stack)

Uygulamaya henüz giriş yapmamış veya sistemsel düzeyde yetkisi tanımlanmamış ziyaretçilerin etkileşime geçmek zorunda olduğu güvenli dış katmandır.

- **LoginScreen ve RegisterScreen:** Her iki ekran da arka planda Firebase Authentication metodlarını asenkron olarak çağırarak kullanıcının JWT erişim anahtarlarıyla güvenli biçimde uygulamanın ana yetkili mekanizmasına alınmasını sağlayacaktır.
- **ForgotPasswordScreen:** Kullanıcının erişimini kaybettiği durumda şifre kurtarma protokollerinin ve doğrulama maillerinin pratik olarak talep edilebildiği parola yenileme arayüzü olarak kurgulanacaktır.

3.1.2 Vatandaş Deneyimi Modülü (Citizen Stack)

Sistemde aktif ve doğrulanmış üyeliği bulunan vatandaşın eylemlerini pratikçe yerine getireceği ana etkileşim menüsüdür.

- **CreateComplaintScreen:** Özel cihaz API'leri yardımı sayesinde kameradan anlık kanıt amaçlı fotoğraf/video çekebilen veya galeriden resim seçilerek sisteme pürüzsüz entegre edebilen dinamik şikayet giriş formu ekranı olarak çalışacaktır.
- **MyComplaintsScreen:** Kullanıcının önceden kamu kurumlarına aktardığı tüm geçmiş şikayetlerin "FlatList" gibi yüksek performanslı React Native listeleyici mimarileri ile ekrana kusursuzca dizileceği kaydırılabilir ekran. Şikayetin o anki kurumsal durumuna (beklemede, işlem aşamasında, tamamlandı vb.) ve gönderim zamanına göre yerel sıralama/filtreleme becerileri desteklenecektir.

- **ProfileScreen:** Vatandaşın toplumsal olaylara yaptığı katkıların ölçüldüğü, uygulamanın oyunlaştırma yönünü de yansıtan motivasyon sayfasıdır. Toplam başvuru sayısı, başarı/deneyim ilerleme çubuğu (seviye barı) ve kazanılmış tecrübe rozetlerinin şık tasarımlarla vurgulandığı kişisel performans arayüzü olacaktır.

3.1.3 Kurum ve Moderatör Paneli Modülü

Sistemin diğer tarafındaki aktör olan kamu görevlileri için geliştirilecek ve yönlendirilen şikayetlere profesyonel geri bildirim sunmak üzere dizayn edilecek modüldür.

- **InstitutionDashboardScreen:** İlgili kuruma (Ör: Ulaşım Dairesi, Sağlık İl Birimi vb.) özel yöneltilmiş güncel başvuruların (ticket yapıları) ve iş yükü metriklerinin özeti sergilenenecektir. Akıllı veri tabanı kuralları sayesinde, Firestore üzerinden yalnızca kuruma özel olan resmi dokümanlar canlı (real-time data listener) biçimde dinlenecek ve kuruma anlık ekran akışı sunulacaktır.
- **ComplaintDetailScreen:** Vatandaşın ilettiği kanıtların detaylı boyutta incelendiği ve gerektiğinde başvuruya "Çözüldü", "Reddedildi" veya "İşleme Alındı" gibi geri bildirim kodlarının (status updates) anında düşüldüğü ve sisteme işlendiği interaktif değerlendirme ekranıdır.

3.2 Veri Tabanı Şeması Tasarımı

Yapılandırılacak modern NoSQL veritabanında (Firestore) en yüksek sorgu hızı ve arama esnekliği sağlanması amacıyla Koleksiyonlara Dayalı (Collection/Document) veri ağacı takip edilecektir. Uygulamanın en temel mantıksal ayırma kurgusu şu yapıda gerçekleştirilecektir:

- **Users (Kullanıcılar) Koleksiyonu:** Platformdaki her kayıtlı organik birimi; UID (Firestore'in sağladığı Benzersiz Kimlik), isim, e-posta formatı, kazanılmış sistemsel tecrübe puanı (XP), edinilmiş dijital rozetler ve yönetimsel rol (admin/vatandaş/kurum/moderatör) detaylarıyla öznitelleştirerek tutan çekirdek yapıdır.
- **Complaints (Şikayetler) Koleksiyonu:** Sistemin kalbi sayılan bu koleksiyon, kategorize edilmiş tüm aktif/pasif şikayet metinlerini, başvurunun atıldığı konum noktalarını (enlem/boylam objesi) ve başvuruya dahil edilmiş medya içeriklerinin Storage URL linklerini eşsiz bir doküman ID'sine bağlı taşıyan ana şemettir.

- **Institutions (Kurumlar) Koleksiyonu:** Uygulama hizmet yelpazesine kayıtlı ve entegre durumdaki yerel kamu kurumlarının/birimlerinin (Belediyeler vb.) genel açık bilgilerini, sistem koordinatörlüğünü ve resmi iletişim kanallarını listeleyen kalıcı kayıt ağacıdır.